

가시화 프로토타입 설명서

2026-09-04 개정 — 탭이 없어졌습니다. 무대는 노드 캔버스 하나입니다.

탭 다섯을 합쳤습니다(요구사항정의서 v1.2 § 3.1). 이식해 온 화면들은 사라지지 않고 **뷰 노드**가 되어 캔버스 안으로 들어왔습니다. 팔레트에서 꺼내 태스크에 이르면 **그 태스크의 장비·구역·시간이 자동으로 그 노드의 조회 범위가** 됩니다.

	옛 화면	지금
뷰 노드들	상단 탭	뷰 노드 넷 — 팔레트에서 꺼내 태스크에 잇는다
캔버스	상단 탭	캔버스 자체 (실행 노드)
화면 전환	탭 클릭	없다. 노드를 더블클릭하면 확대 (오버레이, 동시 하나)
대상·시간 고르기	화면마다 손으로	연결선 하나가 넘긴다
접속 주소	빌드에 박힘	상단 바 ↔ 연결 관리 (VZ-C-07 · 2026-09-04)

아래 § 2 의 화면 설명은 그대로 유효합니다 — 내용은 안 바뀌고 놓이는 자리만 바뀌었습니다. 각 화면은 이제 **접힌 요약 카드**로 먼저 보이고, 더블클릭하면 여기 적힌 전체 화면이 오버레이로 펼쳐집니다.

왜 바꿨는지는 요구사항정의서.md § 3.1 「연결이 곧 범위다」와 HCI_관련연구_차별점_260831.md § 8 차별점 2에 있습니다.

2026-08-28 개정 — 이식이 끝났습니다. 이제 한 앱입니다.

아래 화면 일곱 개는 viz-debugger/ 의 탭 여섯으로 옮겨졌습니다(그 뒤 탭⑥을 빼 지금은 다섯입니다). 10월 예정이었으나 앞당겼습니다 — 시연에서 사이트를 둘 띄우면 “통합했다”가 화면으로 증명되지 않기 때문입니다.

옛 화면	지금 어디에
구역 현황판	탭② (+ 옛 판단·알림의 위험도·표시 깊이가 위쪽에 붙었습니다)
제어 패널	탭③
임무 승인·진행	탭① 안으로 통폐합 (2026-09-01 — 승인만 남기고 구간 진행은 마일스톤이 대신)
지표 조회	탭④
영상 오버레이	탭⑤
판단·알림	해체 — 위험도·표시 깊이→탭②, AI 실패→상단 공통 알림, 자체 관측→공유 계층
노드 그래프	탭⑥ 삭제(2026-08-31) — 노드 에디터 방향이 탭①로 확정되면서 남은 절반도 제거했습니다

web-dashboard/ 는 폐기하지 않습니다. 손대지 않은 **기준선**으로 계속 살아 있고 5173+8787에서 그대로 뜯니다. 통합 앱(5174+8790)과 **동시에 띄워 비교**할 수 있습니다 — 문제가 생겼을 때 “내가 깐는지 원래 그랬는지”를 가르는 유일한 기준이기 때문입니다.

아래 각 화면 설명은 **동작 자체가 바뀌지 않았으므로 그대로 유효**합니다. 통합 후 구조의 원천은 `요구사항정의서.md § 3.1` 입니다.

2026-08-25 개정 — 화면 7개를 기준으로 판단·알림과 노드 그래프를 추가했습니다. 노드 그래프의 F4 계약 무손실 왕복, 등록처 기반 카탈로그, 반영 그래프 역방향 관측과 목 실행기 경계를 반영했습니다. 개정 전 PDF는 `_archive/`에 있습니다.

0. 이게뭔가

요구사항이 말로만 맞는지, 실제로 돌려도 맞는지를 확인하기 위한 프로토타입입니다. 화면을 예쁘게 만드는 것이 목적이 아니라, 요구사항 시트에 적은 주기·상태·경로가 코드로 옮겼을 때도 성립하는지를 보는 것이 목적입니다.

왜 지금 만들 수 있었나 — 목(mock) 게이트웨이

다른 파트의 구현이 아직 없습니다. 그런데 기다리면 가시화는 아무것도 못 만듭니다. 그래서 백엔드 게이트웨이가 있어야 할 자리에 **가짜 서버**를 하나 세웠습니다.

이 가짜 서버는 브라우저 안에 있지 않습니다. **별도 프로세스의 WebSocket 서버**로 돌아갑니다. 이게 중요한 이유는 —

- 브라우저 안에 가짜 데이터를 두면 **네트워크를 타지 않아서** 재연결·구독 복원·지연 같은 것을 검증할 수 없습니다
- 별도 프로세스로 두면 진짜 백엔드가 나왔을 때 **접속 주소만 바꾸면** 됩니다

그리고 이 가짜 서버는 **요구사항 시트의 주기 숫자를 그대로** 발행합니다. 로봇은 임무 중 50ms, 센서는 평시 1분, 카메라는 15fps, 액추에이터는 동작 중 100ms. 시트에 적은 숫자가 그냥 숫자가 아니라 **실제로 그 간격으로 데이터가 오는지**를 볼 수 있습니다.

무엇이 진짜고 무엇이 가짜인가

진짜	가짜
화면 코드 전부	장치에서 오는 값
데이터 레이어 (구독·병합·재연결·상태 보관)	백엔드의 판정과 처리
WebSocket 왕복·HTTP 질의	영상 (합성 도형)
주기·지연·상태 전이 규칙	명령의 실제 물리 실행
파이프라인 계약 직렬화·검증	파이프라인 시험 실행과 하류 노드 전달값

경계가 분명합니다. 가짜인 부분은 전부 목 서버 안에 있고, 화면 코드에는 가짜 데이터를 만드는 경로가 없습니다.

1. 커는 방법

준비물

Node.js 22.6 이상이 필요합니다. 목 게이트웨이를 `.ts` 파일 그대로 실행하기 때문입니다.

```
node -v → v22.6.0 이상
```

없으면 <https://nodejs.org> 의 LTS를 설치합니다.

실행 — 통합 앱 (권장)

```
cd viz-debugger
npm install
npm run dev
```

브라우저에서 <http://localhost:5174> 를 엽니다. (`127.0.0.1` 이 아니라 `localhost` 로 접속해야 합니다.)

`npm run dev` 하나로 목 게이트웨이(8790) · 화면(5174) · STT 서비스(8801) 가 같이 뜹니다. 목 게이트웨이는 하나입니다 — 탭 다섯이 같은 연결 하나를 씁니다. STT는 파이썬 환경이 없으면 경고 한 줄만 남기고 나머지는 그대로 뜹니다 (`viz-debugger/stt/README.md`).

따로 띄우려면 `npm run dev:mock` (게이트웨이만) / `npm run dev:web` (화면만) / `npm run dev:stt` (STT만).

실행 — 기준선 (옛 프로토타입)

```
cd web-dashboard
npm install
npm run dev
```

<http://localhost:5173> · 게이트웨이 8787. 통합 앱과 포트가 겹치지 않으므로 동시에 띄울 수 있습니다. 이 앱은 이식 과정에서 한 줄도 고치지 않았고, 검증 5종 + `typecheck` + `build` 가 이식 전후로 똑같이 통과합니다.

안 켜질 때

가장 흔한 실패는 포트 충돌입니다.

```
[mock-gateway] 기동 실패 - 포트 8790 이 이미 사용 중이다.
```

이전 목 게이트웨이가 안 죽고 남아 있는 경우입니다. 터미널을 `Ctrl+C` 없이 닫으면 이렇게 됩니다. `README.md` 의 “안 켜질 때 — 포트 충돌”에 종료 방법이 있습니다.

어디에 붙을지 정하기 — ⇄ 연결 관리 (2026-09-04 신설 · VZ-C-07)

전에는 접속 주소가 빌드할 때 박혀 있었습니다(VITE_GATEWAY_WS 같은 환경변수). 시연장에서 게이트웨이 IP 하나 바꾸겠다고 다시 빌드할 수는 없어서, 화면에서 정하게 했습니다.

상단 바 「알림」 오른쪽 ⇄ 연결 관리 . 대상은 넷입니다.

대상	지금
백엔드 WS 게이트웨이 (WS/HTTP)	실제로 붙습니다 (기본 8790)
STT 서비스	실제로 붙습니다 (기본 8801)
제어 노드	상대가 아직 없습니다 — 자리만 있고 「연결 예정」
디지털 트윈	상대가 아직 없습니다 — 자리만 있고 「연결 예정」

뒤 둘은 주소를 넣어도 붙을 곳이 없습니다. 없는 것을 있는 척하지 않습니다 — 화면의 다른 자리표시와 같은 규칙입니다.

- 바꾸면 끊고 다시 붙습니다. 나가는 길은 여전히 하나입니다 — 주소만 런타임에 바뀝니다.
- 새로고침해도 남습니다. 틀린 주소를 넣어 아무 데도 못 붙게 되면 「기본값으로 되돌리기」 를 누르면 됩니다.
- 상단의 연결 배지는 상태, 이 버튼은 설정입니다. 배지가 빨강다고 여기부터 열지 말고, 먼저 게이트웨이가 떠 있는지 보세요(위 「안 켜질 때」).

상황을 만들어 보는 방법 — 시나리오

그냥 켜두면 평온한 상태만 보입니다. 장애·지연·실패를 보려면 시나리오를 재생합니다.

```
npm run scenario <이름>
```

예: `npm run scenario camera-silence` → 60초 뒤 카메라가 “판단 불가”로 바뀝니다.

전체 목록은 5장에 있습니다.

1-A. 화면에 뜨는 것 중 무엇이 진짜인가 (2026-08-31 신설)

앱을 그냥 띄우면 남이 줄 데이터 자리가 비어 있지 않고 「연결 예정」 카드로 차 있습니다. 빈칸이 아니라 카드인 이유는, 빈칸은 “우리가 안 만들었다”로 읽히고 카드는 “못 받았다 · 누가 주면 된다”로 읽히기 때문입니다.

카드마다 넷이 적혀 있습니다.

[연결 예정] 영상 스트림 연결 예정	
무엇	관제용 영상 픽셀과 프레임 식별자
누가 보내나	하드웨어 HW-R-07 관제용 고해상도 영상(온디맨드)
	하드웨어 HW-S-06 고정 비전센서(CCTV) 영상
	AI AI-C-08 미디어 입력 관리
	AI AI-C-14 데이터 유형별 경로 분리
우리 자리	백엔드 BE-T-05 사설 IP 라우팅·프록시 중계
	VZ-I-06
	미디어 평면 - 업무·관측 브로커에 실지 않는다

「우리 자리」가 적혀 있는 것이 중요합니다. 그 자리가 있다는 것은 받으면 그릴 준비가 되어 있다는 뜻이고, 못 만든 것이 아니라 못 받은 것이라는 증명입니다.

상대가 아직 없으면 「상대 없음 — 회의 안건」이라고 붉게 적혀 있습니다. 지금은 0건입니다 — 유일했던 한 건(탭⑥ 파이프라인 시험 실행기)은 2026-08-31에 기능을 없애는 쪽으로 결정돼 화면에서 함께 사라졌습니다.

무엇이 자리표시이고 무엇이 우리 목인가

	무엇	화면에서
남이 줄 데이터	영상·탐지·궤적 · 장치 상태·배터리 · 위험도 · 지표 시계열 · 감사 이력 · 액추에이터 상태 · 명령 결과 · 레지스트리 · 역할·권한 · 노드 카탈로그	자리표시 (19곳)
우리가 만드는 목	임무 시나리오 MSN-260826-01 · 마일스톤·태스크 DAG · 8상태 전이 · 되감기 · 실패 경로 격리 · 결함 주입	그대로 그린다
실제로 동작하는 것	음성 인식(STT) · 명령 발행 경로 · 재접속과 구독 복원 · 계약 직렬화 검증	진짜다

임무 시나리오를 자리표시로 바꾸지 않은 이유가 중요합니다. 그건 “아직 못 받은 데이터”가 아니라 논문의 측정 대상입니다. 결함 주입 60건으로 축소율·재현율을 재려면 근본 원인을 아는 합성 데이터라야 하고, 여기에 자리표시를 넣으면 실험이 없어집니다.

다만 임무는 우리 것, 장비 실측값은 남의 것이라는 선으로 갈랐습니다. 그래서 캔버스 안에서도 액션 팝업의 로봇 상태 스트림(배터리·RSSI·관절 온도…)과 배정 풀 카드의 실측 3행은 자리표시입니다.

모드 스위치 — 「목·개발」은 시연 중에 켜지 마세요

(2026-08-31 사이트 개선) 상단 바 우측에 모드 [일반] [시나리오 ▼] [목·개발] 스위치와 ? 버튼이 있습니다. 지금 무엇을 보고 있는지가 세그먼트로 보이고, 화면에 상주하던 설명 문단은 ? (지금 캔버스에 놓인 노드들의 설명서 팝업)로 들어갔습니다. 시나리오 ▼로 대본을 고르면 정지 미리보기(t=0 화면)로 들어가고 — 재생은 여전히 승인(VZ-U-07) 뒤입니다. 개발 도구 (시나리오 재생 버튼·계약 확인)는 목·개발 모드에서만 보입니다.

시나리오 모드에서는 뷰 노드도 그 대본의 값으로 바뀝니다. 지표 노드는 대본이 미는 축에 맞춰 기본 지표가 자동으로 바뀌고(1편 로봇 속도 · 2편 커버리지 · 3편 수위), 대본이 다루지 않는 축의 자리는 「연결 예정」이 아니라 「이 대본에는 해당 없음」으로 갈립니다 — 아래 자리표시 네 상태를 보세요.

대본이 화면을 끌고 갑니다 (2026-09-01 · 2026-09-04 캔버스로 이관)

8/31까지는 연계가 자리 층에서만 있었습니다 — 1편(로봇 이동)을 틀어도 제어 화면에 수문 제어 제목과 버튼 줄과 감사 표가 그대로 있고 안쪽 칸만 「해당 없음」이었습니다. 로봇 이야기를 보는 사람 앞에 수문이 있는 셈이라, 이제 세 층으로 올렸습니다. 탭이 없어진 뒤에는 첫 층이 팔레트로 옮겨 왔습니다.

층	무엇이	예
팔레트	대본이 안 쓰는 뷰 노드는 흐리게 + 이 대본엔 없음. 막지는 않습니다 — 꺼내서 확인할 수 있습니다	1편에서 「제어」
패널	대본이 그 패널의 축을 몰지 않으면 통째로 접습니다 — 제목·버튼 줄·표까지. 접힌 자리에는 어느 편에서 살아나는지와 「그 대본으로 바꾸기」 버튼이 뜹니다	3편에서 구역 맵(궤적·커버리지가 없는 편)
자리	남은 자리는 그대로(연결 예정 / 대본 합성 / cast 밖 / 해당 없음)	감사 이력은 3편에서 값이 차고 1·2편에서는 패널째 접힙니다

세 편이 쓰는 뷰 노드는 이렇습니다 — 이 표는 대본에서 유도한 것이고 손으로 적지 않았습니다 (src/scenarios/axes.ts 의 축→노드 표 하나 · verify:node-scope 가 화면과 대조합니다).

편	쓰는 뷰 노드	안 쓰는 것
MSN-260831-01 엘리베이터 이동	장치·위험 · 지표 · 영상	제어
MSN-260831-02 사각지대 탐지	장치·위험 · 지표 · 영상	제어
MSN-260831-03 월류방어벽 개폐	장치·위험 · 제어 · 지표	영상

(실행 노드는 임무 축이라 어느 대본에서든 늘 살아 있습니다.)

일반 모드에서는 아무것도 접히지 않습니다. 팔레트도 패널도 그대로 뜨는 것이 「남이 줄 데이터가 어디에 얼마나 있는지」를 보여 주는 화면이기 때문입니다. 목·개발도 전부 그렇습니다.

대본 띠가 「지금 무엇이 어디서 보이는지」를 말합니다. 상단 띠 둘째 줄에 재생 머리 기준 진행 중인 노드와 갈 노드 버튼이 뜹니다 — 예: 지금: MS-C close_gate 발행 [제어 노드로] . 그 노드가 캔버스에 아직 없으면 누를 때 만들어져 진행 중인 태스크에 이어집니다. 이미 있으면 하이라이트됩니다 — 팔레트를 몰라도 이 버튼이 가르쳐 줍니다.

시나리오 중에도 자리표시로 남는 셋과 그 이유는 ? 오버레이에 적혀 있습니다 — 배정 풀의 실측 3행(VZ-D-07 · 남이 줄 데이터라 지어내지 않습니다) · 임무 이력(백엔드 보관) · 레지스트리(백엔드가 주는 목록).

목·개발 을 켜면 예전처럼 목이 그려집니다. 개발과 화면 확인용입니다.

켜면 지워지지 않는 배지가 뜹니다. 화면 맨 위에 붉은 띠가 깔리고, 목으로 그려진 자리마다 목 — 실제 데이터 아님 표가 붙습니다. 끄는 방법은 모드 스위치에서 일반 을 고르는 것뿐입니다 — 켜 것을 잊고 시연하면 원래 문제(진짜처럼 보이는 목)로 되돌아가기 때문입니다.

목 게이트웨이는 끄지 않았습니다. 여전히 8790에서 발행하고 데이터 계층도 여전히 받습니다. 바뀐 것은 그리는 층뿐입니다. 그래서 재접속·구독 복원·캐시 정책 검증은 그대로 돌아갑니다.

시연 순서 — 문장 세 개 (2026-08-31 · 시나리오 대본)

발화 하나로 탭 다섯이 같은 이야기를 하는 것을 봅니다. 캔버스의 발화 패널에서 아래 문장을 녹음하거나 「문장을 직접 넣기」 로 입력하고, 승인하면 재생이 시작됩니다. 이것은 LLM이 아니라 키워드 대조로 미리 써 둔 대본을 꺼내는 것입니다 — 발화 패널의 뒤 세 칸이 목 에서 대본 배지로 바뀌고, 어느 키워드가 맞았는지가 그 자리에 뜹니다.

문장	대본	승인 후 어느 노드에서 무엇이 움직이나
「503호에서 복도에 있는 엘리베이터 앞까지 이동해줘.」	MSN-260831-01	캔버스 마일스톤 7건 재생 · 장치·위험 노드 카드의 robot-01 위치·속도 · 지표 노드 로봇 속도 시계열 · 영상 노드 문을 지나는 동안만 탐지 박스
「503호 내에서 고정 카메라의 사각지대를 탐지해줘.」	MSN-260831-02	장치·위험 노드 구역 맵 미니뷰 — 사각지대 두 칸이 채워지고 칸마다 마지막 탐지 시각 · 10분(600초) 초과분 재탐색이 마일스톤 화면에 파생 2회차로 · 지표 노드 커버리지 % · 영상 노드 시야 밖(사각지대)에서는 박스 없음
「월류방어벽 자동 개폐 시스템을 가동해.」	MSN-260831-03	지표 노드 수위 곡선 + 위험 수위 선 · 제어 노드 close_gate → open_gate 4단계 (발행 주체 = 임무) · 장치·위험 노드 수위·개도율 100→0→100

- 승인 전에는 아무것도 재생되지 않습니다 — 매칭 결과는 제안이고, 마일스톤 화면에 제안 상태로만 뜹니다.
- 「안녕하세요」 처럼 맞는 대본이 없는 문장은 거부됩니다(no_script_match · 상단 알림). 비슷한 것을 억지로 고르지 않습니다.
- 옛 문장 「415호에서 503호로 이동해줘」 도 포함니다 — 다만 그 편은 registry 세계와 연결되지 않은 구판 세계라 뷰 노드들에 아무것도 따라 움직이지 않고, 화면에 그렇게 안내가 뜹니다.

scenario 모드 배지 — 목·개발 띠와 다릅니다

scenario 모드로 들어가는 길은 둘입니다 (2026-08-31 사이트 개선).

- 승인 자동 — 발화가 대본에 맞고 승인(VZ-U-07)하면 재생과 함께 들어갑니다. 세그먼트에 시나리오 · 260831-0N 재생 중 .

- **수동 미리보기** — 모드 스위치의 **시나리오** ▾ 에서 대본을 고릅니다. 대본의 **t=0 화면만** 그리고 **재생하지 않습니다**. 세그먼트에 **시나리오 · 260831-0N** 정지. 「그린다」 까지가 미리보기이고 「재생한다」 는 여전히 승인 뒤입니다 — 스위치가 승인 선을 우회하지 않습니다.

들어가면 화면 맨 위에 **초록 띠** 한 줄이 깔립니다 — 「대본 · MSN-... · **합성 데이터** · 등장 장비 N대」 . **나오는 길은 모드 스위치의 일반 하나**이고, 자리표시(placeholder)로 복귀하면서 게이트웨이의 재생·미리보기도 함께 닫힙니다. 재생이 끝나면 「재생 끝 — 마지막 상태 유지」 로 바뀝니다.

- **대본에 등장하는 장비(cast)만** 그려집니다. 장치·위험 노드의 카드도 카드 단위로 깔립니다 — **robot-02 · sensor-04** 처럼 대본에 안 나오는 장비는 대본 중에도 여전히 자리표시입니다. 그래야 「이 값은 대본이 준 것」 과 「이 자리는 아직 아무도 안 준 것」 이 깔립니다.
- **대본이 다루지 않는 축의** 자리는 「이 대본에는 해당 없음」 으로 갈리고, 어느 편에서 보이는지가 같이 뜹니다 — 1·2편의 수문 자리, 3편의 영상 자리가 그렇습니다. 자리가 비어 보이는 이유가 「못 받았다」 가 아니라 「이 대본의 이야기에 없다」 임을 구별하기 위한 것입니다.
- 모드를 **목·개발** 로 바꾸면 붉은 띠가 대본 띠 위에 겹칩니다(목이 이깁니다 — 전부 그려집니다).

그래서 자리표시는 네 가지로 갈립니다 — ① **연결 예정** (일반 모드 · 아직 아무도 안 준 자리) · ② **대본 — 합성 데이터** (대본 등장 장비) · ③ **이 대본에 없는 장비** (cast 밖) · ④ **이 대본에는 해당 없음** (대본이 그 축을 몰지 않음).

2. 캔버스 하나 — 무엇을 보면 되나

화면을 옮기는 조작이 없습니다. **마일스톤 목록** → **노드 캔버스** 두 층이고, 그 안에서 노드를 더블클릭해 확대합니다.

마일스톤 목록 ← 최상위
 ↳ 노드 캔버스 ← 무대는 여기 하나

팔레트 [장치·위험] [제어] [지표] [영상]
 실행 노드 — 마일스톤 → 태스크 → 액션 아이템
 뷰 노드 — 팔레트에서 꺼내 태스크에 이은 것
 확대 — 더블클릭. 오버레이라 캔버스가 뒤에 남습니다

구독은 화면이 아니라 데이터 계층이 들고 있습니다. 노드를 열고 닫아도 끊기지 않습니다 — 그 계층은 앱 수명과 같습니다 (옛 탭 시절부터 같은 규칙입니다).

뷰 노드를 잇는다는 것

팔레트에서 노드를 꺼내 태스크에 이르면 넷이 자동으로 넘어갑니다.

넘어가는 것	어디서
대상 장비	그 태스크의 target
구역	레지스트리 (VZ-I-03)
시간 창	그 태스크의 실행 구간
재생 머리	되감기(VZ-D-04) — 캔버스 전체가 같은 시각을 봅니다

탭 시절에는 “수문 A의 지표를 보려면 탭④에 가서 대상을 고르고 시간을 고른다”였습니다. 지금은 **선 하나를 이으면 끝입니다.**

연결하지 않은 「전역 노드」도 됩니다. 구역 전체를 볼 때 씁니다. 연결선이 없고 전역 배치가 붙어서 이 값이 이 태스크 것인지 전체 것인지 헷갈리지 않습니다.

아래는 그 넷의 내용입니다. 화면 내용은 탭 시절과 같습니다 — 놓이는 자리만 바뀌었습니다.

실행 노드 그래프 - 노드 문법 다섯과 되돌아가는 화살표

(2026-08-31 사이트 개선) 그 전에는 **마일스톤 하나에 태스크가 하나**여서, 마일스톤을 눌러도 그래프에 노드가 하나뿐이었습니다. 대본 세 편의 태스크를 **노드 문법 다섯**으로 폈습니다 — 관측(sense) · 판정(decide) · 구동(act) · 검증(verify) · 보고(report) . 노드 위에 그 칩이 붙습니다.

대본	마일스톤	노드
MSN-260831-01 엘리베이터 이동	7	7 → 17
MSN-260831-02 사각지대 탐지	6	7 → 18
MSN-260831-03 월류방어벽 개폐	4	4 → 16

단계를 더하거나 뺀 것이 아니라 한 단계 안을 편 것입니다. 실패 분기를 지어 넣지 않았고, 마일스톤의 시작·끝 시각을 그대로 두어 되감기 눈금과 뷰 노드들의 시간축 정렬이 바뀌지 않습니다. 옛 편 MSN-260826-01 (HCI 전달본)은 손대지 않았습니다 — 전달본 화면은 그대로입니다.

- **[이 마일스톤] [임무 전체] 범위 토글** — 머리줄 오른쪽에 있습니다. 분기와 합류는 대부분 마일스톤을 건너기 때문입니다(1편의 합류는 MS-D→MS-E, 2편의 되돌아감은 MS-F→MS-C). 기본은 마일스톤이라 전달본의 화면 흐름은 그대로입니다. (2026-09-04 — 그 왼쪽에 있던 **DAG/트리 토글**은 **없어졌습니다**. 회의록 § 10의 「그래프 형태 vs 트리 형태」가 **그래프로** 단혔기 때문입니다(요구사항정의서 § 7.10). 대신 머리줄이 그 임무 자신의 모양을 적습니다 — **태스크 DAG** — 17노드 · 합류 1. 트리와의 비교는 화면이 아니라 측정 도구(`npm run measure:representation`)와 논문에서 다룹니다.)
- **되돌아가는 화살표(↩)**는 점선입니다. 2편의 재탐색 루프(T-27c → T-23a)가 그것입니다. 일부러 의존(deps)과 갈라 두었습니다 — 의존에 넣으면 깊이 계산이 순환에 빠집니다. 그리기만 하고 배치에는 넣지 않습니다. 마일스톤 밖으로 나가는 되돌아감이 있으면 그래프 아래에 한 줄로 알리고 「임무 전체로 보기」 버튼이 붙습니다.

되돌아가는 길 — 이동 경로 (2026-09-01)

그래프 머리줄 맨 위에 **이동 경로**가 생겼습니다.

마일스톤 > MS-C 위험 수위 도달 시 닫기 > T-43c close_gate 발행
↳ 누르면 마일스톤 목록 ↳ 액션 팝업이 열렸을 때만 눌립니다 (→ 그래프)

- 맨 앞 「마일스톤」은 **항상 있고 항상 눌립니다**. 그 전에는 그래프에서 마일스톤으로 돌아가는 길이 상단 공통 바의 임무 이름 버튼 하나뿐이었는데, 그게 되돌아가기라는 것을 화면 어디에도 적어 두지 않았습니다 — **발견할 수 없는 길은 없는 길입니다**. (상단 바의 그 버튼은 그대로 있습니다. 단독 빌드에서도 같은 자리입니다.)
- 가운데 칸은 지금 보고 있는 마일스톤이고, 「임무 전체」 범위일 때는 **임무 전체 N노드**입니다. 그래프 화면에서는 **이미 보고 있는 자리라 눌러지 않습니다**(굵은 글자). 액션 아이템 팝업이 열려 있을 때만 버튼이 되어 그래프로 돌아갑니다 — 눌러는 것처럼 보이면 거짓말이 됩니다.
- 마지막 칸은 액션 아이템 팝업이 열려 있을 때만 나옵니다.
- 되감기·실패 강조 화면에도 같은 줄이 나옵니다 — 그 둘도 똑같이 갇혀 있었습니다.
- 「마일스톤」으로 나가면 **범위도 「이 마일스톤」으로 되돌립니다**. 전체로 보다 나갔다 다시 들어왔는데 전체로 남아 있으면 어리둥절하기 때문입니다.

화면 폭에 맞춰 접습니다 (2026-09-01)

「임무 전체」는 거의 일직선 사슬이라 깊이가 그대로 열 수가 됐고, 깊이 하나당 열 하나를 무조건 오른쪽에 붙이던 배치에서는 **폭이 3,300~3,550 px** — 화면의 두세 배가 됐습니다. 첫 화면에서 오른쪽 노드가 보이지 않았고 페이지가 옆으로 늘어났습니다.

이제 그래프가 **자기 자리의 실제 폭을 재서 밴드로 접습니다**. 한 밴드에 들어갈 열 수를 그 폭에서 구하고, 넘치는 깊이는 다음 밴드의 첫 열로 내립니다 — 세로만 길어지고 가로는 화면 안에 남습니다. 노드를 끌어 옮기는 것은 그대로이고, **창 크기가 바뀌어도 옮긴 노드는 되돌아가지 않습니다**.

자리 폭	한 밴드	세 편의 「임무 전체」
1,088 px (1180px 최소 창)	4열	4밴드 × 4열 — 폭 870 px · 높이 825~1,125 px

자리 폭	한 밴드	세 편의 「임무 전체」
1,828 px (1920px 창)	8열	2밴드 × 8열 — 폭 1,750 px · 높이 385~685 px

밴드를 넘어가는 연결선은 **실선 파랑에** **줄바꿈** 표시가 붙습니다 — 되돌아가는 참조 엣지(**점선 주황**)와 헛갈리면 안 되기 때문입니다. 하나는 「줄바꿈」 이고 하나는 「루프」 입니다.

2-1. 장치·위험 뷰 노드 (옛 탭② 구역 현황판)

지금 어디에 — 팔레트의 「장치·위험」. 접으면 연결/배터리/자기보고 3층 한 줄 + 위험도 등급이고, 더블클릭하면 아래 화면이 펼쳐집니다. 옛 「판단·알림」의 위험도 카드와 표시 깊이 3단(결심자·운영자·개발자)이 여기 함께 있습니다 (VZ-U-03 · v1.1).

관련 요구사항 VZ-U-01 · VZ-I-01 · VZ-I-02 · VZ-I-03

무엇을 보는 화면인가 — 구역 안의 장치가 지금 어떤 상태인지를 카드로 늘어놓습니다.

여기서 확인할 것은 상태가 넷이라는 점입니다.

표시	뜻	어떻게 나오나
정상	잘 돌아감	기기 자기보고 ok + 서버가 online 판정
장애	고장	기기가 스스로 fault를 보고
의도적 미배포	원래 없음	배포되지 않은 대상
판단 불가	모름	서버가 값을 오래 못 받음 (stale)

“장애”와 “판단 불가”를 가르는 것이 이 화면의 핵심입니다. 둘을 하나로 합치면 “고장난 것”과 “연락이 안 되는 것”이 같아 보이는데, 관제에서 이 둘은 완전히 다른 조치를 부릅니다.

그래서 상태를 하나로 뭉쳐서 저장하지 않고 **세 층(기기 자기보고 / 서버 판정 / 배포 여부)**을 각각 보관하고 표시값만 파생시킵니다.

꼭 봐야 할 것

- robot-03 — 값을 한 번도 발행하지 않는데 카드가 보입니다. 레지스트리(존재해야 할 목록)를 근거로 그리기 때문입니다. 값이 오는 것만 그리면 “있어야 할 게 안 왔다”를 영영 못 봅니다
- 새로고침 — 빈 카드가 없습니다. 구독하는 순간 백엔드가 마지막 값을 한 번 던져주기 때문입니다. 이게 없으면 센서는 최대 1분간 빈 화면입니다
- `npm run scenario camera-silence` → 60초 뒤 **판단 불가**로 바뀝니다. 이때 기기 자기보고는 마지막 값(ok)이 그대로 남아 있습니다 — 두 층이 서로 다른 주체의 말이라는 증거입니다

2-2. 제어 뷰 노드 (옛 탭③ 제어 패널)

지금 어디에 — 팔레트의 「제어」. 접으면 이 태스크가 낸 명령 수와 마지막 명령의 4단계 위치입니다. 명령은 여전히 단일 출구를 거칩니다 — 실행 노드의 재실행 버튼과 같은 길입니다.

관련 요구사항 VZ-0-01 · VZ-0-02 · VZ-0-05 · VZ-I-05 · VZ-C-04

무엇을 보는 화면인가 — 수문을 열고 닫고, 그 결과가 어떻게 돌아오는지를 봅니다.

여기서 확인할 것은 “눌렀다”와 “됐다”가 다르다는 점입니다.

명령은 네 단계로 돌아옵니다 — 수신 확인 → 수행 중 → 물리 상태 변화 → 완료. 화면은 이걸 세 가지(진행중·확정·실패)로 보여주되, 수신 확인을 확정으로 치지 않습니다. 수문은 되돌리기 어려우니 “접수했다”가 아니라 “실제로 움직였다”를 확인해야 합니다.

두 개의 키

명령을 누르면 화면이 요청 식별자를 붙여 보내고, 백엔드가 상관키를 발급해 내려줍니다. 왜 돌아오냐면 —

- 상관키는 백엔드가 만드는데, 그게 도착하기 전 구간에도 버튼을 잠그고 진행 표시를 걸어야 합니다
- 그 구간에 걸 대상이 요청 식별자입니다

화면에는 이 두 키가 하나의 상태로 보입니다. 키가 몇 개인지는 데이터 레이어 안에서 끝납니다.

꼭 봐야 할 것

- `npm run scenario ack-late` — 상관키가 진행 이벤트보다 늦게 도착합니다. 그 사이에 온 단계들이 보류됐다가 흡수되고, “보류 후 흡수” 표시가 붙습니다. 순서를 믿는 코드는 실제 백엔드에서 깨지기 때문에 일부러 만든 경로입니다
- `npm run scenario ack-drop` — 상관키가 끝내 안 옵니다. 화면은 “만료 — 실행 여부를 이 화면이 알 수 없다”고 정직하게 표시합니다
- `npm run scenario control-lock` — 통신이 끊기면 버튼이 잠기고 사유가 보입니다. `control-unlock` 후에도 서버가 실제 상태를 재확인할 때까지 잠금이 유지됩니다
- `npm run scenario role-narrow` — 역할을 503 구역 담당으로 좁히면 다른 구역 수문의 버튼이 잠깁니다. 화면에 “잠금 우회해 발행” 버튼이 있는데, 눌러 보면 서버가 거부합니다. 화면 차단은 편의일 뿐 실제 방어선은 백엔드라는 것을 눈으로 보는 장치입니다
- 마지막 조작자 패널 — 열 때만 조회하고 주기 폴링을 하지 않습니다. 개발자 도구 네트워크 탭에서 확인할 수 있습니다

2-3. 임무 승인 — 마일스톤 화면의 제안 카드 안

지금 어디에 — 화면 하나로 남기지 않고 마일스톤 화면의 제안 카드 안입니다. 승인·거부(VZ-U-07)가 마일스톤 목록 위에 붙습니다 (2026-09-01 재배치).

관련 요구사항 VZ-U-07

무엇을 보는 화면인가 — AI가 만든 계획을 사람이 승인합니다.

2026-09-01 (2차) — 결정이 끝나면 한 줄 영수증으로 접힙니다. 승인된 뒤에도 근거 4층과 산출 경로가 화면 한가운데를 차지해 마일스톤을 아래로 밀고 있었는데, 그 시점에는 이 화면에서 결정할 것이 하나도 없습니다. 마일스톤 화면의 주인은 마일스톤입니다.

- 승인 전 — 근거 4층을 펼친 채로 보이고 승인·거부 버튼을 받습니다(관문이라 그대로).
- 결정 후 — ✓ 승인됨 · 시각 · 계획 id · [근거 ▼] 한 줄. 펼치면 근거 4층과 산출 경로가 그대로 나옵니다 — 사라지는 것이 아니라 한 번 눌러야 보이는 것으로 바뀝니다.
- 「어떻게 동작하는가」(경로 hop 의 뜻 · AI 구간과 백엔드 중계 구간을 왜 갈라 두는가)는 **우상단 ? 설명서**로 옮겼습니다. 화면에 남는 것은 이 계획의 실측 단계와 시각입니다.

2026-09-01 — 「서브태스크 진행」(구간 목록) 패널을 지웠습니다. 계획 구간은 마일스톤과 **같은 값**이라(재생기가 태스크 상태를 접어 구간에 되돌려 줍니다) 한 화면에서 같은 것을 두 번 그리고 있었습니다. VZ-U-05 는 없어진 것이 아니라 **마일스톤 목록 · 태스크 그래프 · 되감기 타임라인이 충족**한다고 보고, 그 판단을 요구사항정의서 § 3.1에 적었습니다. 실패 상세도 실패 경로 강조 화면과 겹쳐 지웠고, 중계 단계 표시는 **목·개발** 모드로 내렸습니다.

여기서 확인할 것은 승인 전에는 아무것도 실행되지 않는다는 점입니다.

AI가 계획을 만들고 검증까지 마쳐도, **사람이 승인 버튼을 누르기 전까지 로봇은 움직이지 않습니다**. 목 서버가 승인 없이는 진행 이벤트를 아예 보내지 않도록 만들어 놔습니다.

근거를 펼쳐 볼 수 있습니다 — 어떤 임무에서 나왔는지, 어느 구역을 어떤 순서로 지나는지, 어떤 검증을 통과했는지, 어느 버전의 생성기가 만들었는지.

누가 가져다주는가 — 계획을 만드는 것은 AI지만, **가시화에 가져다주고 승인 결과를 받아 가는 것은 백엔드**입니다. 화면에 중계 경로가 표시되어 AI 산출 구간과 백엔드 중계 구간이 구분됩니다. 나중에 승인이 안 먹었을 때 “AI가 계획을 못 만든 것”인지 “중계가 끊긴 것”인지를 갈라야 하기 때문입니다.

꼭 봐야 할 것

- `npm run scenario plan-propose` — 계획이 승인 대기로 내려옵니다. 승인하면 **한 박자 뒤에** 실행이 시작됩니다 (백엔드 중계 구간). 거부하면 사유가 같은 경로로 돌아갑니다
- **결정하기 전에는** 근거 4층을 접지 않습니다 — 승인은 되돌리기 어려운 조작이라 「펼쳐 봐야 보이는 근거」는 안 보는 근거가 됩니다. 다만 그중 「구간별 계획」 절은 아래 마일스톤 목록과 같은 값이라 **한 줄**(「N구간 — 아래 마일스톤과 같음」)로 줄였습니다
- **결정이 끝난 뒤에는** 접습니다. 그때 펼쳐 두는 것은 근거를 보이는 일이 아니라 자리를 차지하는 일입니다
- 실패한 구간이 어느 단계에서 왜 멈췄는지는 **실패 경로 강조 화면**에서 봅니다

2-4. 지표 뷰 노드 (옛 탭④ 지표 조회)

지금 어디에 — 팔레트의 「지표」. 점으면 미니 스파크라인과 요약/원본 표기입니다.

관련 요구사항 VZ-I-04 · VZ-C-03

무엇을 보는 화면인가 — 수위·유속 같은 시계열을 그래프로 봅니다.

여기서 확인할 것은 평시에 받는 값이 원본이 아니라는 점입니다.

설계상 raw는 엣지에 남고 백엔드에는 구역 요약만 올라옵니다. 장치가 늘어도 백엔드 부하가 장치 수가 아니라 구역 수에 비례하게 만드는 구조입니다.

그래서 화면에 오는 값에는 “요약 · 구역 · 15초” 같은 표기가 붙습니다. 이 표기가 없으면 화면이 요약값을 원본으로 알고 또 평균을 냅니다. 그러면 숫자가 틀리는데 그래프는 멀쩡해 보입니다. 발견이 늦는 종류의 오류라 아예 계산을 막아 댔습니다.

원본이 필요하면 따로 요청합니다. “원본 보기”로 전환하면 질의 프록시가 엣지로 중계해 가져옵니다 — 느립니다. 실측으로 요약은 12ms에 60점, 원본은 975ms에 900점이었습니다. 이 느림은 연출이 아니라 실제 엣지 중계를 흉내 낸 것이고, 화면이 “가져오는 중”을 그릴 이유가 여기서 나옵니다.

꼭 봐야 할 것

- 그래프 위의 **요약 표기** — 지금 보는 게 요약인지 원본인지
- 요약값에 평균을 적용해 보면 **계산이 수행되지 않고** 차단 사유가 뜹니다
- `npm run scenario agg-unlabeled` — 집약 표기가 **깨진 채로** 옵니다. 화면은 이걸 원본으로 단정하지 않고 “**표기 불명**”으로 두고 계산을 막습니다. 모르는 것을 원본으로 취급하면 보호 장치가 조용히 꺼지기 때문입니다
- `npm run scenario agg-normal` — 정상 표기로 되돌립니다

2-5. 영상 뷰 노드 (엣 탭⑤ 영상 오버레이)

지금 어디에 — 팔레트의 「영상」. 접혀 있으면 정지 프레임이고 확대해야 재생됩니다(VZ-I-06) — 노드를 셋 놓아도 프레임 루프가 셋 돌지 않게 하기 위한 계약입니다.

관련 요구사항 VZ-I-06 · VZ-I-07 · VZ-I-09

무엇을 보는 화면인가 — 영상 위에 AI 탐지 결과(박스)를 겹쳐 그립니다.

이 화면이 프로토타입에서 값이 제일 큼니다. 그림으로만 주장하던 것을 숫자로 바꿔 놓았기 때문입니다.

문제는 이렇습니다. AI가 프레임을 보고 결과를 내는 데 시간이 걸립니다. 0.2초 걸린다면, 결과가 도착할 때 화면은 이미 3프레임쯤 앞서 있습니다(15fps 기준). 그 결과를 그냥 지금 화면에 그리면 박스가 대상을 못 따라갑니다.

해결은 결과에 “이건 몇 번 프레임을 본 결과다”를 실어 보내는 것입니다. 화면은 그 프레임에 맞춰 그립니다.

정합 on/off 토글이 있습니다.

설정	엣지 정밀 인지	온디바이스 최소 안전 판단
정합 켜	0.0px	0.0px
정합 끄 · 지연 200ms	91.8px (평균 57.7)	33.6px (평균 18.4)
정합 끄 · 지연 500ms	210.0px (평균 136.4)	30.7px (평균 18.2)

2026-08-24 재측정 (커밋 09add2c 기준). 앞선 판의 46.8px·102.3px는 **인지 출처를 나누기 전** 수치라 지금 화면과 맞지 않습니다.

지연을 2.5배로 늘리면 엷지 쪽 어긋남이 2.3배가 됩니다. 프레임 참조가 왜 계약에 필요한지를 이 표 하나로 설명할 수 있습니다.

온디바이스 값이 지연과 무관하게 30px 안팎인 이유는 온보드 판단이 60ms로 고정돼 있기 때문입니다. `vision-delay-`*는 **엷지 추론 지연만** 바꿉니다 — 안전 판단이 엷지 왕복을 기다리지 않는다는 것이 여기서도 드러납니다.

꼭 봐야 할 것

- 토글을 꺾다 켜보면 박스가 대상에서 뒤쳐지는 것이 **눈에 보이고**, 뒤쳐진 픽셀 거리가 화면에 숫자로 뜹니다
- `npm run scenario vision-delay-500` — 어긋남이 커집니다
- `npm run scenario vision-bbox-normalized / vision-bbox-absolute` — 좌표 형식을 바꿔도 **박스가 제자리에** 옵니다
- 신뢰도가 낮은 탐지는 **시각적으로 구분**됩니다. 확실한 것과 애매한 것이 똑같이 보이면 안 되니까요
- 온디바이스의 진행영역·접근 변화와 엷지의 정밀 분류·추적은 **출처별로 다른 색과 표기**를 씁니다
- `vision-edge-off`에서는 기본 온디바이스 판단은 남고 정밀 인지만 사라집니다. `vision-link-off`에서는 관측 소스별 추적을 유지합니다
- 참조 프레임이 버퍼에 없으면 **참조 프레임 없음**을 표시하며, 현재 프레임과 비교한 수치를 정합 결과로 주장하지 않습니다
- **점으면 정지 프레임이 됩니다.** 안 보는 영상을 계속 그리면 렌더 예산을 낭비합니다

영상은 합성입니다. 실제 픽셀은 업무·관측과 분리된 미디어 평면으로 가야 합니다(AI-C-14). GStreamer 어댑터의 AI 입력 경로는 생겼지만 뷰어용 출력 분기와 담당이 아직 정해지지 않아, 목 서버가 프레임 번호가 찍힌 도형 영상을 15fps로 내려줍니다. 정합 검증 목적에는 충분합니다.

하드웨어 카드 더블클릭 — 대상 상태 조회 (2026-09-04 · `VZ-D-07`)

지금 어디에 — 마일스톤 화면 오른쪽 **하드웨어 패널**. 카드를 더블클릭하면 열립니다.

전에는 카드를 마일스톤에 **끌어다 배정**하는 것만 됐습니다. `VZ-D-07` 은 배정과 **상태 조회**를 함께 요구하는데 조회 쪽이 비어 있었습니다.

더블클릭하면 그 대상의 상태가 오버레이로 열립니다 — 액션 아이템 팝업이 쓰는 것과 **같은 부품**입니다(배터리·네트워크·IP·펌웨어·관절 온도·하트비트). 두 곳에 다른 상태 화면을 만들면 어휘가 갈리기 때문입니다.

대본을 재생 중이라면 여기 실측값이 뜨지 않습니다. 대본 세계의 장비 상태는 **남이 줄 데이터**라 지어내지 않습니다(8/31 결정) — 대신 「무엇을 · 누구에게서 기다리는가」가 적힌 자리표시가 뜹니다. **그게 이 화면의 요지입니다.**

카메라 연결 상태는 아직 계약에 없습니다. 자리만 만들어 두고 「연결 예정」으로 둡니다 — `Hardware` 계약에 그 필드가 없고, 하드웨어 파트와 맞춘 뒤에 넣습니다 (요구사항정의서 § 7.10 「열림」).

2-6. 해체됨 · 옛 판단·알림

지금 어디에 — 화면으로 남지 않았습니다. 위험도 근거와 표시 깊이는 **장치·위험 뷰 노드**로, AI 실패는 **상단 공통 알림**으로(그 노드를 보고 있지 않아도 알아야 하므로), 자체 관측은 공유 계층으로 갔습니다.

관련 요구사항 VZ-I-08 · VZ-I-10 · VZ-O-04 · VZ-U-03

무엇을 보는 화면인가 — AI 판단의 결과만 보여주는 것이 아니라, 어떤 입력과 규칙으로 그 판단에 도달했는지와 가시화 자체가 제때 처리되고 있는지를 함께 봅니다.

같은 데이터를 보더라도 역할에 따라 필요한 깊이가 다릅니다. 운영자는 지금 조치할 내용을, 분석자는 근거와 규칙을, 결심자는 권고와 영향 범위를 먼저 봅니다. 화면이나 역할을 바꾼다고 구독을 새로 만들지 않고 **표시 깊이만 바꿉니다**.

꼭 봐야 할 것

- 위험 단계와 권고 옆의 **근거 펼치기** — 입력값·규칙·판정 사유를 되짚을 수 있습니다
- AI 결과가 없거나 실패했을 때 정상처럼 빈칸으로 두지 않고 **판단 불가·실패 상태**를 드러냅니다
- 클라이언트 처리 지연은 장치 상태와 섞지 않고 별도 자체 관측 지표로 보냅니다
- 경고는 달을 수 있지만 원인이 해소되지 않은 사실까지 지워지지 않습니다

2-7. ~~탭⑥~~ 파이프라인 편집기 (2026-08-31 제거)

지금 어디에 — 통합 앱에서 제거됐습니다. 8/28 이식 때 임무 관제 모드를 지우고 파이프라인 모드만 남겼다가, 2026-08-31에 노드 에디터의 구현 방향이 **캔버스**로 확정되면서(전회의) 나머지도 제거했습니다. 요구사항 2건(VZ-U-04 · VZ-L-04)도 시트에서 삭제했습니다.

아래 설명은 **기준선 web-dashboard (5173)의 노드 그래프 탭** 기준으로만 유효합니다 — 그쪽은 그대로 동작합니다.

관련 요구사항 VZ-U-04 = REQ-1001 ~ REQ-1007

무엇을 보는 화면인가 — 기본 모드에서는 임무→Sub task→실행·영상·센서 근거를 한 장에 연결하고, 보조 모드에서는 source→transform→sink 데이터 해석 파이프라인을 편집합니다.

임무 관제 모드

계획이 도착한 대상과 같은 구역의 근거 후보를 **레지스트리**에서 구성합니다. 특정 robot-01 이나 네 장치 목록을 화면 코드에 고정하지 않으므로 장치와 임무 대상이 늘면 선택지와 근거 노드가 함께 늘어납니다. Sub task의 대기·진행·완료·실패·건너뛸 상태와 근거 연결을 한 화면에서 볼 수 있습니다.

데이터 파이프라인 모드

- 노드를 놓고 포트를 연결하면 방향·타입·필수 입출력·순환·고립 여부를 검사합니다
- 초안은 { pipeline, layout } 입니다. pipeline 은 contracts/pipeline.schema.json 그대로이고 좌표는 계약을 오염시키지 않도록 바깥 layout 에 둡니다

- **계약 JSON 보기**에서 좌표 없는 F4 계약을 확인할 수 있습니다. 같은 계약을 다시 읽으면 별도 레이아웃과 합쳐 같은 화면으로 복원됩니다
- 노드 목록은 코드 상수가 아니라 `contracts/examples/` 의 등록 디스크립터·렌더러·데이터소스에서 만들어집니다. 예시 파일을 하나 추가하면 TypeScript를 고치지 않아도 카탈로그가 늘어납니다
- 시험 실행 뒤 받은 토큰이 있어야 운영에 반영할 수 있고, 반영 뒤에는 직전 버전으로 되돌릴 수 있습니다

시험 결과와 운영 관측은 다릅니다. 시험 결과의 `rows.elapsed_ms` 는 초안에 대한 **목 실행 결과**입니다. 별도 **운영 그래프 관측** 표는 반영된 그래프의 노드별 최근 수신 시각·건수·마지막 값과 붙은 대상을 보여줍니다. `source`는 게이트웨이가 실제로 받은 목 장치 봉투를 관측하지만 `transform.sink`는 목 실행기가 전달한 값입니다. 화면도 둘을 다른 영역과 문구로 구분하며, 실물 실행기나 영속 관측인 것처럼 보이지 않습니다.

꼭 봐야 할 것

- 임무 관제에서 정상 임무와 4번 Sub task 실패 임무를 재생해 진행·실패 전파 확인
 - 데이터 파이프라인에서 잘못된 연결을 시도해 구체적인 거부 사유 확인
 - **계약 JSON 보기**에서 노드 좌표가 계약 본문에 없는지 확인
 - 시험 실행 → 운영 반영 뒤 **모의 시험 결과와 운영 그래프 관측**이 분리되어 있는지 확인
 - 새 시험 없이 바뀐 초안을 반영할 수 없는지, 직전 버전 되돌리기가 되는지 확인
-

3. 눈에 안 보이지만 중요한 것

화면에 나타나지 않지만 요구사항의 핵심인 것들입니다.

3-1. 데이터는 전량 받고, 화면 반영만 묶는다

로봇이 초당 20번 값을 보냅니다. 받을 때마다 화면을 다시 그리면 **오히려 버벅입니다**. 그렇다고 데이터를 버리면 이동 궤적이 끊깁니다.

그래서 데이터는 **전부 받되 화면 반영만 100ms 창으로 묶습니다**. 초당 10회를 넘지 않습니다.

3-2. “값이 오래됐다”는 판정은 서버가 한다

화면이 마지막 수신 시각으로 계산하면 **사용자 PC 시계에 의존**하게 됩니다. 시계가 틀린 PC에서는 멀쩡한 장치가 판단 불가로 뜹니다. 그래서 판정은 서버가 하고 화면은 결과만 받습니다.

3-3. 감사는 화면이 쓰지 않는다

“누가 언제 무엇을 시켰나”는 화면이 직접 기록하지 않고 **필드만 실어 보냅니다**. 이유가 셋입니다 —

1. **위조** — 화면이 직접 쓰면 “누가 했다”를 클라이언트가 자기 입으로 말하는 셈입니다
2. **시계** — 장치는 NTP로 시각을 맞추지만 사용자 PC에는 그런 보장이 없습니다
3. **유실** — 별도 경로로 보내면 화면을 닫는 순간 “실행은 됐는데 기록이 없음”이 생깁니다

3-4. 끊기면 스스로 붙고, 구독을 되살린다

목 서버를 껐다 켜면 화면이 **자동으로 재접속**하고 원래 구독을 복원합니다. 복원되면 현재값이 다시 채워져서 화면에 공백이 없습니다.

4. 지금 확인할 수 있는 것 / 아직 없는 것

확인할 수 있는 것

- 상태 4종이 실제로 구분돼 표시되는가 → **된다**
- 요구사항에 적은 주기대로 데이터가 오는가 → **온다** (50ms · 1분 · 15fps · 100ms · 15초)
- 프레임 참조가 없으면 박스가 얼마나 어긋나는가 → **숫자로 나온다** (엣지 200ms에 91.8px · 500ms에 210.0px)
- 승인 없이 계획이 실행되는가 → **안 된다**
- 권한 범위 밖 명령이 화면을 우회해도 막히는가 → **막힌다**
- 요약값을 다시 평균 내려 하면 어떻게 되는가 → **계산이 수행되지 않는다**
- 서버를 껐다 켜면 화면이 복구되는가 → **된다**
- 재접속 때 캐시 금지 채널이 재생되지 않고 원본 시각이 보존되는가 → **자동 검증을 통과했다**
- 같은 게이트웨이 계약으로 3D 공간에 위치를 그리는가 → **Unity 뷰어에서 된다**
- (아래 셋은 **기준선 web-dashboard (5173)에서만 확인합니다** — 탭⑥ 제거로 통합 앱에는 없습니다)
 - 노드 편집기 초안이 좌표 없이 F4 계약으로 왕복하는가 → **자동 검증을 통과했다**
 - 등록 파일만 추가해도 노드 카탈로그가 늘어나는가 → **자동 검증을 통과했다**
 - 반영된 파이프라인에서 지금 흐르는 값을 노드별로 볼 수 있는가 → **된다** (하류는 목 전달값)
- 탭 다섯이 **게이트웨이 하나로** 다 차는가 → **찬다** (`npm run verify:one-gateway` 자동 검증 통과)
- **단독 빌드**가 뷰 노드 코드를 끌어오지 않는가 → **안 끌어온다** (`verify:standalone` 통과)
- 명령 출구가 하나인가 → **하나다** (`verify:single-egress` 통과. 4단계 추적과 감사 필드가 살아 있다)

아직 없는 것

없는 것	이유
음성 명령	2026-08-28 구현됨. 발화 패널에서 마이크로 말하면 실제로 인식되고 결과를 고칠 수 있습니다. 신뢰도 임계는 아직 잠정입니다 (<code>reports/2026-08-28_1620_STT이식.md</code>)
LLM 그래프 초안 생성	사람 편집·계약 검증·시험·반영 경로는 구현됨. 의도 해석 계약 확정 뒤 같은 절차 앞에 연결
실제 카메라 영상	미디어 평면은 확정. 뷰어용 출력 분기·담당은 미정
실제 백엔드·파이프라인 실행기 연결	현재 카탈로그·시험 실행·하류 관측은 목. 계약과 HTTP 경계를 유지해 교체하도록 구성
Unity 시뮬레이터	현재 Unity는 계약 왕복을 검증하는 뷰어. 원천 교체는 2단계

5. 시나리오 전체 목록

npm run scenario <이름>

이름	무엇이 일어나나
camera-silence	camera-02 침묵 → 60초 뒤 판단 불가
camera-resume	camera-02 발행 재개 → online 복귀
sensor-offline	sensor-02 연결 끊김 → offline 후 복구
sensor-surge	sensor-01 급변 → 즉시 발행 + 이벤트 모드(1초)
command-fail	다음 명령 1건을 실패시킴
ack-late	상관키를 진행 이벤트보다 늦게 보냄 → 보류 후 흡수
ack-drop	상관키를 아예 안 보냄 → 만료 정리
agg-unlabeled	집약 표기에서 종류를 빼고 발행 → 표기 불명 처리
agg-odd-string	집약 표기를 계약 밖 형태로 발행
agg-normal	집약 표기를 정식 계약값으로 되돌림
role-narrow	역할을 503 구역 담당으로 좁힘
role-full	역할을 전 구역으로 되돌림
control-lock	actuator-01 통신 두절 → 제어 잠금
control-unlock	actuator-01 복구 → 재확인 후 해제
plan-propose	계획 하나를 승인 대기로 내려보냄
plan-propose-failing	구간 4/5에서 실패하는 계획
vision-delay-200	추론 지연 200ms (기본)
vision-delay-500	추론 지연 500ms → 어긋남 확대
vision-bbox-normalized	bbox를 정규화 좌표(0~1)로
vision-bbox-absolute	bbox를 픽셀 절대 좌표로
vision-inference-320 / vision-inference-640	추론 해상도 전환 — 표시 해상도와 달라도 박스가 제자리인지
robot-idle / robot-mission	robot-01 임무 토글 — 50ms(20Hz) ↔ 5초
vision-edge-off / vision-edge-on	선택형 엣지 정밀 인지를 제거/복구
vision-link-off / vision-link-on	다중 관측 연계를 제거/복구
cache-policy-audit	캐시 정책 선언과 실제 캐시를 대조
command-roundtrip-slow / command-roundtrip-zero	명령 왕복 지연 주입/해제

6. 처음 보는 사람을 위한 5분 코스

1. `npm run dev` → `http://localhost:5173`
2. 구역 현황판 — `robot-03` 이 값 없이도 카드로 있는 것 확인. 새로고침해도 빈 카드가 없는 것 확인
3. `npm run scenario camera-silence` → 60초 기다렸다가 판단 불가 확인
4. 제어 패널 — 수문 열기. 진행중 → 확정으로 바뀌는 것과, 수행 중 진행 상태가 보이는 것 확인
5. `npm run scenario role-narrow` → 다른 구역 수문 버튼이 잠기는 것 확인. “잠금 우회해 발행” 눌러서 서버가 거부하는 것 확인
6. 영상 오버레이 — 정합 토글을 껐다 켜면서 뒤쳐진 픽셀 수가 화면에 뜨는 것 확인
7. `npm run scenario vision-delay-500` → 그 숫자가 커지는 것 확인
8. 판단·알림 — 역할을 바꿔도 구독은 유지되고 설명 깊이만 달라지는지 확인
9. 노드 그래프 — 임무 실패 전파를 본 뒤 데이터 파이프라인에서 시험 실행→반영→운영 관측 확인

6·7번은 영상 정합의 필요성을 숫자로 보여주고, 9번은 계약·편집·운영 관측의 경계를 한 번에 보여줍니다.